

Copy Constructor

A **Copy Constructor** is a special constructor in C++ that initializes a brand-new object by making an exact copy of an existing object of the same class.

It is the specific mechanism that powers "Pass-by-Value" for custom objects.

When is the Copy Constructor called?

The compiler automatically invokes the copy constructor in three specific scenarios:

1. **Explicit Initialization:** When you create a new object and immediately set it equal to an existing one.

```
Car car1("BMW");  
Car car2 = car1; // Copy constructor called!  
Car car3(car1); // Alternative syntax, copy constructor called!
```

2. **Passing an object by Value:** When you pass an object into a function without using a reference (&) or a pointer (*).

```
void myCar(Car c) { ... }  
myCar(car1); // Copy constructor called to make the local 'd' copy!
```

3. **Returning an object by Value:** When a function returns a local object. (Note: Modern C++ compilers often optimize this away using Return Value Optimization (RVO), but conceptually, it relies on the copy constructor.

The Syntax and Signature

A copy constructor always follows a very strict signature. It takes a **constant reference** to an object of its own class.

```
ClassName(const ClassName& other);
```

- **const:** Ensures that the original object you are copying from cannot be accidentally modified during the copy process.
- **& (Reference): CRITICAL.** You must pass the original object by reference. If you tried to pass it by value, C++ would have to call the copy constructor to make a copy for the parameter, which would call the copy constructor again, creating an infinite loop that crashes the compiler.

Why do you need to write one?

If you do not write a copy constructor, C++ automatically generates a default one for you. However, the default copy constructor only performs a **shallow copy**.

Copy Assignment Operator

The **Copy Assignment Operator** (operator=) is used to copy the contents of one **already existing** object into another **already existing** object. While the Copy Constructor is for **birth** (initialization), the Copy Assignment Operator is for **updates** (replacement).

1. The Syntax

```
ClassName &operator=(const ClassName &other);
```

2. The Steps for a Safe Assignment

1. **Self-Assignment Check:** Check if the source is the same as the target (if (this == &other)). If you skip this, and you do c1 = c1, you might delete your own data before you copy it!
2. **Deallocate Old Memory:** Clean up the resources the current object is already holding to avoid a memory leak.
3. **Allocate and Copy (Deep Copy):** Create new memory and copy the values from the other object.
4. **Return *this:** Return the current object to support chaining.

```
class Student {
private:
    char *name;
    int regno;

public:
    // 1. Constructor
    Student(const char *name, int regno) : regno(regno) {
        this->name = new char[strlen(name) + 1];
        strcpy(this->name, name);
    }

    // 2. Destructor
    ~Student() { delete[] name; }

    // 3. Copy Constructor (For initialization: Student s2 = s1)
    Student(const Student &s) : regno(s.regno) {
        name = new char[strlen(s.name) + 1];
        strcpy(name, s.name);
    }

    // 4. Copy Assignment Operator (For existing objects: s2 = s1)
    Student &operator=(const Student &s) {

        // Self-assignment check
        if (this == &s) { return *this; }
```

```

        delete[] name; // Free existing resource to avoid memory leak

        // Deep copy
        this->regno = s.regno;
        this->name = new char[strlen(s.name) + 1];
        strcpy(this->name, s.name);

        return *this;
    }
};

int main() {

    Student s1("Ismail", 1); // Creating s1

    Student s2 = s1; // Using Copy Constructor

    Student s3("Temporary", 0); // Creating s3 with different data

    s3 = s1; // Using Copy Assignment Operator

    return 0;
}

```

4. Copy Constructor vs. Copy Assignment

Copy Constructor: Create a new object from an already existing object. **Car c2 = c1;**

Copy Assignment: Replace an already existing object with an already existing object **c2 = c1;** (c2 already existed from a previous line; we are just changing its value).

The "Copy-and-Swap"

In professional C++, we often use a more advanced pattern called **Copy-and-Swap**. It handles the self-assignment check and memory safety automatically by using a temporary copy and then swapping the internals. **(will see later)**